

Session 2: Persistent Data Structures

Block 2 — Introduction to OCaml

Léon Gondelman

September 9, 2026

SW7 Programming Paradigms Course

Aalborg University @ Copenhagen

Block 2: Introduction to OCaml

A general-purpose language, *statically and strongly typed*, from the ML family (with Standard ML and F#).

- Designed and implemented at Inria, in France: Caml from 1985, and OCaml since 1996, by **Xavier Leroy**, Damien Doligez, Jérôme Vouillon, Didier Rémy and others — Leroy's is the course we followed for the first half of today.
- Types are *checked* everywhere but *written* almost nowhere: the compiler infers them, and gives strong guarantees in return.
- Compiled to native code, and garbage collected.
- Immutable by default — yet mutation is available when we ask for it. That last point is why it belongs in this course.

A language that has an opinion, but still lets us choose.

Who writes OCaml

- **Jane Street** — a trading firm whose entire system is OCaml; one of the largest single codebases in any typed functional language.
- **Proof assistants and verification tools:** Rocq (until recently Coq), Why3, Frama-C, Alt-Ergo — the tools used to prove that programs do what they claim.
- **Static analysis:** Astrée, used on Airbus flight-control software; Infer and Flow at Meta; Semgrep.
- **Systems:** the Tezos blockchain, the Unison file synchroniser, MirageOS unikernels.

Not an accident: OCaml turns up where being wrong is expensive.

let: definitions, and the types we never write

```
let x = 40 + 2                (* a value *)

let double n = 2 * n          (* a function: no braces, no return *)

let area r =
  let pi = 3.14159 in         (* a LOCAL binding, in force below *)
  pi *. r *. r                (* *. is multiplication on floats *)
```

There is no return: the value of the body *is* the result.

In the interactive toplevel the compiler tells us what it worked out:

```
# let double n = 2 * n ;;
val double : int -> int = <fun>
```

We wrote no type at all, and yet double is fully typed.

OCaml is not “the functional language”

Mutable state is available — we need only ask for it by name.

```
let counter = ref 0          (* a mutable cell *)

let tick () =
  counter := !counter + 1;    (* := writes, ! reads *)
  !counter
```

tick takes (), the single value of type unit — OCaml’s way of saying *no interesting argument*, and the return type of anything whose purpose is an effect.

The standard library ships a mutable Stack, Queue and Hashtbl beside the immutable List and Map. *The paradigm is a choice made inside the language, not by it.*

Lists: a linked list with the links hidden

```
let nothing = []           (* the empty list *)
let l  = [1; 2; 3]         (* a literal -- note the SEMICOLONS *)
let l' = 1 :: 2 :: 3 :: [] (* exactly the same list *)
```

A list is a chain of cells, each holding a value and a pointer to the rest — the linked list from the first half of today. The pointer is hidden, as in Java and unlike in C; and unlike Java it cannot be assigned either. There is no `.next` to write to, and no assignment for list cells anywhere in the language.

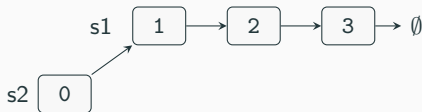
So a list behaves as a Python *tuple* behaves — we may read it and build new ones from it, but we cannot change one. The nested pairs we built by hand in Python are what OCaml calls, without ceremony, a list.

Careful: `[1, 2, 3]` is not an error. It is a list of *one* element, the triple `(1, 2, 3)` — and the complaint arrives later, somewhere else.

Lists: immutable, and shared

```
let s1 = [1; 2; 3]      (* separator is ; not , *)  
let s2 = 0 :: s1        (* [0; 1; 2; 3] *)
```

s2 is one new cell whose tail *is* s1. Nothing was copied, and nothing can be modified: there is no assignment to a list cell in the language at all.



This is the sharing diagram from the first half, now built into the language.

match: asking what shape a value has

```
let rec sum l =  
  match l with  
  | []          -> 0  
  | x :: rest  -> x + sum rest
```

A list is either empty or a head followed by a tail; `match` names those two cases and binds `x` and `rest` in the second.

Two things a `switch` never did for us:

- it *destructures* — `x` and `rest` come out of the value, no accessors needed;
- it is *checked* — omit a case and the compiler says which one. Session 3 makes that guarantee the whole point.

let rec: recursion

A function that calls itself must say so: `let rec`.

```
let rec fact n =  
  if n <= 1 then 1 else n * fact (n - 1)
```

```
let rec length l =  
  match l with  
  | []      -> 0  
  | _ :: tl -> 1 + length tl
```

On numbers we recurse towards zero; on lists we recurse towards the empty list. The shape of the data decides the shape of the function — an idea we take much further in session 3. *Loops are not forbidden; they are not the first thing we reach for.*

Records: the type our queue will have

Given to us in the lab. A record groups several values under names:

```
type 'a t = { front : 'a list; back : 'a list }  
  
let empty = { front = []; back = [] }      (* construction *)  
let is_empty q = q.front = []             (* field access *)
```

And a shorthand worth recognising before we meet it:

```
{ front; back }      (* means { front = front; back = back } *)
```

'a is the type variable — Java's <T>. A record is immutable unless a field is declared mutable, so every operation on this queue necessarily *returns a new one*.

option: OCaml has no null

What should dequeue return on an empty queue? OCaml refuses to let us answer “nothing, silently”.

```
val dequeue : 'a t -> ('a * 'a t) option
```

A value of type `'a option` is either `None` or `Some v`. The empty case is *in the type*: the value is only reachable by matching, and the compiler reports a match that forgets `None` — in the lab’s dune setup, as an error.

`option` is a type defined by *listing its cases* — our first *algebraic data type*, and the only one we borrow today. Declaring our own is session 3.

Python	<code>raise IndexError</code>
Java	<code>throw new NoSuchElementException</code>
OCaml	<code>None</code> — checked at compile time

The same decision, made three times; only one of them is enforced.

A record may be mutable — and the paradigm changes

A field marked mutable can be assigned with `<-`. Here is the same stack, written imperatively:

```
type 'a stack = { mutable elems : 'a list }

let create () = { elems = [] }
let push v s  = s.elems <- v :: s.elems
let pop s =
  match s.elems with
  | []          -> None
  | v :: rest   -> s.elems <- rest; Some v
```

Same language, same data, same operations. The whole difference is in one type:

imperative	<code>val push : 'a -> 'a stack -> unit</code>
persistent	<code>val push : 'a -> 'a t -> 'a t</code>

unit says: I changed something we cannot see.

Running OCaml from a terminal

Interactively — `utop` is friendlier, if installed:

```
$ ocaml  
# 1 + 2 ;;  
- : int = 3
```

As a script, for a single file:

```
$ ocaml tour.ml
```

Compiled, which is what we normally want:

```
$ ocamlfind ocamlopt -package str hello.ml -o hello  
$ ./hello
```

`ocamlopt` produces native code; `ocamlc` produces bytecode. For more than two files this becomes tedious quickly — which is the point of the next slide.

tour.ml in the lab repository is this part of the lecture, as a program we can run.

dune: the build system we will actually use

A Makefile would work. dune is what the OCaml world settled on, and it needs almost no configuration — two small files describe the whole lab:

```
(* dune-project *)      (lang dune 3.0)

(* lib/dune *)          (library (name pqueue_lib) (modules pqueue))
```

Three commands are all we need today — compile, compile and test, and run:

```
$ dune build
$ dune runtest
$ dune exec bin/solver.exe -- ../mazes/medium.txt --bfs
```

dune works out the dependencies between files itself; there is no list of source files to maintain.

.ml and .mli: the interface is the contract

Each module may carry an interface. The .mli says what exists; the .ml says how.

```
(* pqueue.mli -- the contract *)
type 'a t                (* abstract -- no definition here *)

val empty    : 'a t
val enqueue  : 'a -> 'a t -> 'a t
val dequeue  : 'a t -> ('a * 'a t) option
```

Because 'a t is given *without* a definition, no client can see that a queue is two lists, and therefore no client can build one by hand or depend on the representation. We are free to change it tomorrow.

The compiler checks the .ml against the .mli. This is the whole of today's first half, said in one file — and session 11 returns to it properly.

What we have not said yet

Today was the reading knowledge needed for the lab. The rest arrives when there is a reason for it.

Idea	Today	Properly in
Sum types, exhaustiveness	option only	session 3
Functions as values, fun	mentioned	session 4
Recursion and tail calls	two examples	session 4
map, filter, fold, closures	used once, unexplained	session 5
Lazy evaluation, Seq	the cliffhanger	session 5
Mutation, and when it is safe	ref	session 7
Polymorphism, 'a, inference	'a as <T>	session 10
Modules, signatures, functors	one .mli	session 11

Enough OCaml to build a queue is a surprisingly small language.

Lab Session and Homework

Lab session, and what to do afterwards

Warm-up, on paper. The queue's invariant, and why the $O(n)$ reversal is affordable.

The quest, at the keyboard.

- *In Python:* we implement it, then drive the maze solver — breadth-first, the one-line exchange to depth-first, the history view.
- *In OCaml:* the same queue again, against a given `.mli`.
- *And then we break it:* one version dequeued repeatedly redoes the same $O(n)$ reversal every time. Session 5 makes that result shared.

Homework.

- Finish the quest, and write the queue a third time in **Java**.
- Small **OCaml exercises** — lists and pattern matching only.
- The **proofs** on the paper sheet: the invariant, the FIFO argument, and the assumption persistence breaks.